

■ Design Shopify / Multi-Tenant E-Commerce

System Design Interview | E-Commerce & Marketplace

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Merchants create stores; Manage products/inventory/orders; Custom domains; Themes; Payment processing; Analytics dashboard
Non-Functional	1M+ merchants; Each merchant isolated; BFCM: 10× traffic spike; Tenant isolation (no data leakage); 99.99% uptime
Scale	Shopify: \$9.3B GMV on BFCM 2023; 500M+ storefronts globally; Must scale per-tenant and globally simultaneously

■ Multi-Tenancy Architecture

- Shared infrastructure, isolated data: all merchants on same cluster but data strictly partitioned by shop_id
- Database sharding: shops distributed across DB shards via consistent hashing on shop_id; each shard serves ~10K shops
- Pod architecture (Shopify's approach): group of shops assigned to a 'pod' (isolated compute + DB cluster); reduces blast radius of failures
- Storefront delivery: Liquid templates + CDN; each merchant's storefront cached at edge keyed by shop_id + URL

■ Tenant Isolation Strategies

- **Row-Level:** All tables include shop_id; every query WHERE shop_id = current_tenant; risk: missing WHERE clause leaks data; mitigate via middleware
- **Schema-Per-Tenant:** Separate DB schema per tenant; better isolation; harder to query cross-tenant; scales to ~10K tenants
- **DB-Per-Tenant:** Dedicated DB per tenant; maximum isolation; expensive; used only for enterprise/large merchants
- **Shopify approach:** Row-level isolation with app-level enforcement + connection pooling per shard; middleware injects shop_id on every query

■ Traffic Spike Handling (BFCM)

- Storefront is read-heavy: 95%+ traffic is browsing (not checkout); aggressive CDN caching for product pages, collection pages
- Cache-busting: when merchant updates product → invalidate CDN cache for that shop_id's URLs; Fastly instant purge API
- Checkout isolation: checkout service separately scaled; dedicated capacity reserved per merchant based on historical peaks
- Queue-based checkout: under extreme load, queue checkout requests and process sequentially per merchant; preserve order integrity

■ Data Model (Sharded by shop_id)

- shops: shop_id(PK), domain, plan, created_at, settings — master table on dedicated shard
- products: product_id, shop_id, title, price, inventory_qty, variants[] — sharded by shop_id
- orders: order_id, shop_id, customer_id, line_items[], total, status, created_at — sharded by shop_id
- Cross-shard queries: avoid by denormalizing; use separate analytics DB (BigQuery) for cross-merchant reporting

■ Multi-Tenancy Models

Shared DB (Row Isolation)	Dedicated DB per Tenant
✓ Cost efficient	✓ Maximum isolation
✓ Easy deployment	✓ Easier compliance (GDPR)
✓ Hard data leakage to prevent	✓ High cost
✓ Cross-tenant analytics easy	✓ Used for enterprise tiers

■ Payment & Revenue Share

- Shopify Payments (built-in): Stripe under the hood; 0% transaction fee vs 0.5-2% for external gateways
- Payment routing: each merchant configured gateway; checkout service selects correct gateway per shop_id
- Revenue share tracking: every order event published to Kafka → analytics pipeline → billing system computes fees
- Payout: batch job aggregates merchant earnings; transfers via ACH/wire based on payout schedule (daily/weekly)

■ Merchant Analytics

- Real-time dashboard: order count, revenue, top products; Kafka → Flink → Redis for live counters
- Historical analytics: order events → data warehouse (BigQuery/Snowflake); pre-aggregated daily/weekly/monthly
- Tenant isolation in analytics: each merchant only queries their own data; row-level security in DW enforced via service layer
- Benchmarks: Shopify shows merchants their rank vs industry average; anonymized cross-merchant aggregate stats